Demand-to-Delivery Diagnostic Agent

Domain: Supply Chain

Three-Day Capstone Project | 24 Hours Total | VS Code + Jupyter/IPython | Python 3.11 / 3.12

Core technologies: Neo4j knowledge graph · OpenAI generative AI · Python · open-source retrieval & evaluation libraries

Training scenario notice

This project brief is a synthetic, representative enterprise scenario created for the AGCO Advanced AI Bootcamp capstone. It reflects the program's published transformation themes without implying access to, or disclosure of, AGCO confidential operational data or production systems. Every dataset referenced in this brief is synthetic and will be generated separately by the program team.

Document Control

Program	AGCO Advanced AI Bootcamp — Capstone Program
Applies to	Foundation, Practitioner, and Advanced bootcamp tracks
Format	Three-day team capstone, 8 hours/day (24 hours total)
Environment	VS Code with a Jupyter/IPython kernel · Python 3.11 or 3.12
Document status	v2.0 — restructured capstone brief

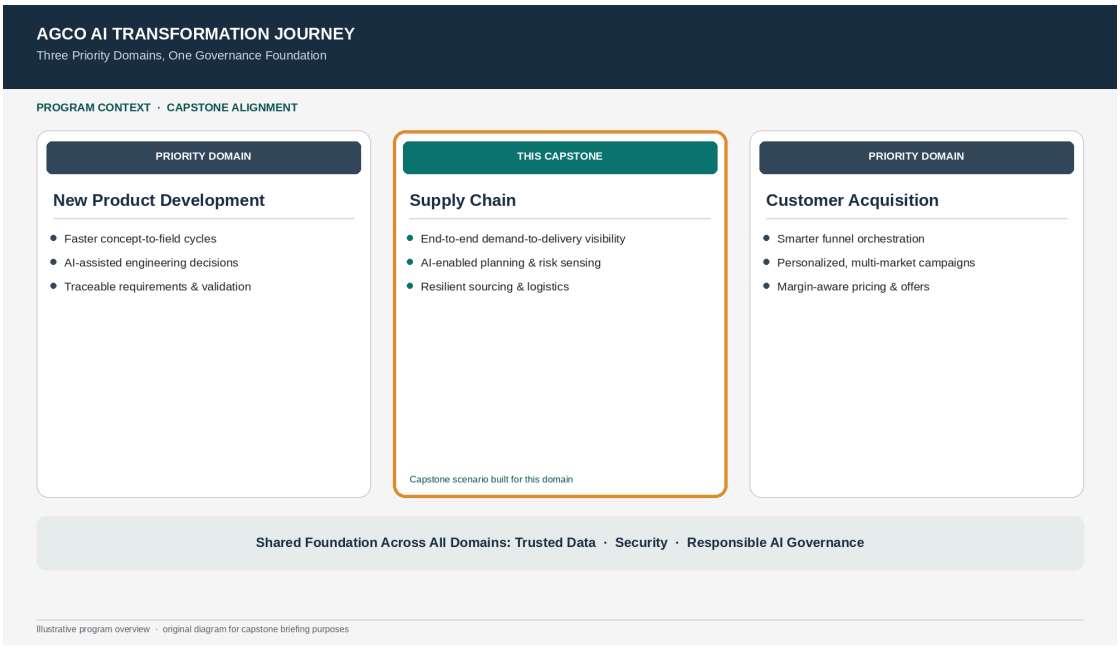
Contents

1. Capstone at a Glance

- **Build an incremental knowledge graph** — extract entities and relationships, create deterministic IDs, and prevent duplicates during repeated ingestion.
- **Translate natural language into graph operations** — map user intent to Neo4j/Cypher-driven traversals that retrieve structured evidence.
- **Use hybrid retrieval** — combine semantic/vector retrieval with graph traversal to produce multi-hop, evidence-grounded answers.
- **Evaluate the agent** — apply an LLM-as-a-judge together with task-specific quality, latency, and reliability metrics.
- **Demonstrate enterprise governance** — protect secrets and sensitive data, log decisions and evidence, handle uncertainty, and keep high-impact actions under human control.

2. Program Context — AGCO's Three Priority Domains

AGCO's AI transformation program prioritizes three domains — New Product Development, Supply Chain, and Customer Acquisition — built on a shared foundation of trusted data and governance. This capstone turns one common Agentic Knowledge Graph brief into a standalone project for each domain: every track builds the same class of system — an incremental knowledge graph, hybrid retrieval, an evaluated agent, and enterprise guardrails — applied to a different real business problem.



Original program-context diagram prepared for this capstone briefing; the highlighted card marks this document's domain.

3. Target Personas

The agent is built for the following primary users. Each persona brings a different angle on the same underlying evidence.

Persona	Why they use the agent
Demand Planner	Compares forecast, orders, seasonality, and planning variance.
Supply / Materials Planner	Evaluates inventory, purchase orders, shortages, and substitutions.
Supplier / Quality Manager	Investigates supplier capacity, quality holds, and recurring part issues.
Plant / Operations Leader	Needs impact on builds, dates, products, and mitigation choices.

Persona Spotlight

> **Marcus, Demand Planner.** An exception flag appears on a high-use component two weeks before a scheduled build for a harvester platform. Marcus needs to understand quickly whether this is a supplier delay, a forecast shift, a quality hold, or some combination — and what mitigation options exist elsewhere in the network. Today that analysis takes hours across separate planning, procurement, and quality systems — the diagnostic agent should surface a ranked, evidence-backed explanation in minutes.

4. Business Scenario

Business Challenge

Demand planning, supplier capacity, component availability, logistics, inventory, quality events, and plant schedules interact in ways that are difficult to diagnose from isolated spreadsheets. A forecast shift or supplier event can

propagate through multiple products and plants, while alternate inventory or approved substitutions may exist elsewhere in the network.

Scenario Anchor

> A synthetic harvester platform is projected to miss a production target because a high-use component will fall below safety stock. The immediate signal looks like supplier lateness, but the full picture includes a forecast uplift, a quality hold on one batch, inventory at another plant, and an approved substitute with limited compatibility. The team needs one evidence-backed explanation and a prioritized mitigation plan.

5. Problem Statement

> **In one sentence**
>
> Planners and plant leaders cannot quickly explain why a product, plant, or part is at risk — because forecast, supplier, purchase-order, shipment, inventory, quality, and production data live in separate systems, and a single disruption can cascade through multiple products and plants.

What makes this hard today

- A shortage signal can look like supplier lateness when the real driver is a forecast shift, quality hold, or a combination of causes.
- Alternate inventory or approved substitutes elsewhere in the network are hard to discover quickly.
- Mitigation trade-offs — coverage vs. lead time vs. operational risk — are not visible in one place.
- Root-cause analysis today depends on manually joining spreadsheets across teams.

6. Proposed Solution

The capstone solution is a diagnostic agent that lets planners and plant leaders ask why a product, plant, or part is at risk and receive a ranked, evidence-backed explanation with mitigation options — not an isolated report from a single system. It traces the disruption across forecast, supplier, purchase-order, shipment, inventory, quality, and production relationships.

Core capabilities

- Understands natural-language planning questions and plans the right retrieval strategy.
- Traces the causal chain from demand/supply signals to affected products and plants.
- Quantifies the exposed scope using the supplied dataset.
- Surfaces feasible mitigations — substitutes, alternate sources, transfers — with constraints.
- Keeps procurement/production changes under human approval while explaining trade-offs.

7. Product Requirements Document (PRD)

7.1 Functional Requirements

- **FR-1.** The system shall construct and incrementally maintain a Neo4j knowledge graph covering products, parts, suppliers, forecasts, orders, shipments, inventory, and quality events, using MERGE/upsert logic so repeated ingestion does not create duplicate entities.
- **FR-2.** The system shall extract entities and relationships from structured records and unstructured narrative evidence, resolving aliases and repeated mentions before creating new graph nodes.
- **FR-3.** The system shall translate natural-language questions into safe graph retrieval — templated Cypher, validated generated Cypher, or a controlled query planner.
- **FR-4.** The system shall combine vector/semantic retrieval over unstructured evidence with Neo4j graph traversal to support multi-hop, evidence-grounded reasoning.
- **FR-5.** The system shall use an OpenAI-powered agent to plan retrieval, select tools, and synthesize a diagnosis restricted to evidence returned by its own tools rather than model memory.
- **FR-6.** The system shall return a structured response containing a diagnosis, likely causes/drivers with confidence and evidence IDs, evidence paths, affected scope, contradictory/missing evidence, recommended next actions, and an overall confidence score.
- **FR-7.** The system shall include an evaluation harness that scores benchmark questions with an LLM-as-a-judge and objective/task-specific checks, and records latency and failure modes.

7.2 Non-Functional Requirements

- **Reliability** — repeated ingestion of the same source files shall not increase node/relationship counts for entities that already exist (idempotent or near-idempotent loading).
- **Security** — API keys, database credentials, and other secrets shall never be hard-coded in notebooks or source files.
- **Auditability** — every user query, tool call, retrieved evidence ID, and judge result shall be logged in a reviewable format.
- **Explainability & traceability** — every diagnostic conclusion shall be traceable to specific graph paths and/or source evidence identifiers.
- **Performance** — the system shall record end-to-end and per-stage (graph, vector, synthesis) latency, and show at least one measurable improvement after a tuning iteration.
- **Human control** — consequential or high-impact actions shall require explicit human review rather than autonomous execution.

7.3 Scope Boundaries

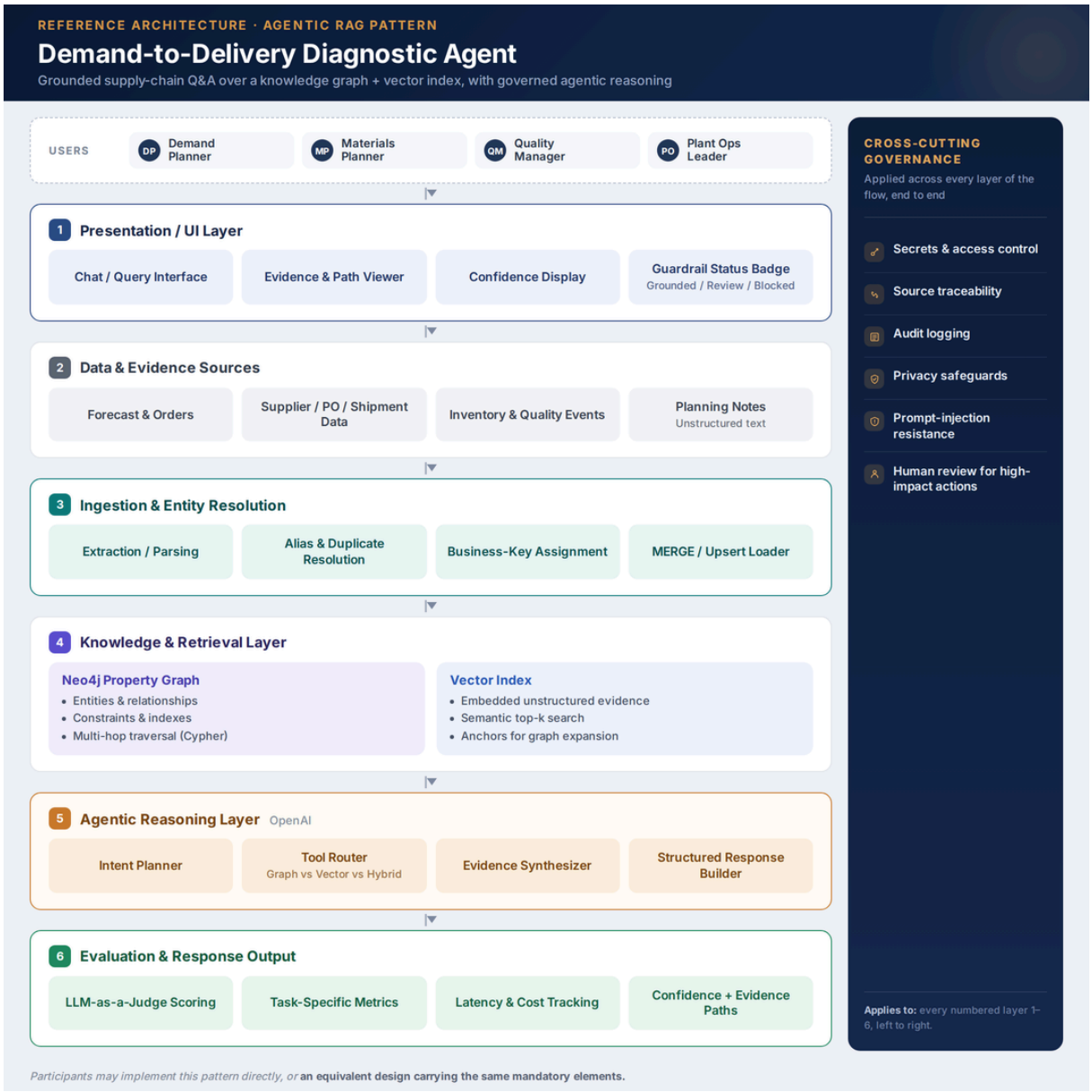
In scope	Out of scope
Synthetic demand, supplier, and logistics dataset provided by the program team. Incremental Neo4j graph of demand-to-delivery relationships. Hybrid diagnostic agent for shortage/disruption root-cause and mitigation analysis. Evaluation harness and governance controls described in this brief. Local VS Code / Jupyter execution environment.	Production deployment into AGCO operational systems, or access to live confidential data. Autonomous high-impact decisions — physical equipment control, purchasing commitments, price changes, or outbound customer actions. Using hidden model knowledge as evidence when the required answer should come from the supplied capstone dataset. Hard-coded API credentials or secrets inside notebooks or source files.

7.4 Success Metrics

Delivery is measured against the rubric in Section 12: Graph Construction & Integration (30%), Hybrid Retrieval & Reasoning (30%), Evaluation & Tuning (20%), and Enterprise Governance (20%).

8. Reference Architecture

Participants may implement this pattern directly or use an equivalent design. The mandatory elements are Neo4j, OpenAI-based generative AI, hybrid graph + vector retrieval, evaluation, and governance.



Layered reference architecture: users → presentation/UI layer → data sources → ingestion → knowledge & retrieval → agentic reasoning → evaluation, with cross-cutting governance.

8.2 Component Responsibilities

Layer	What it does	Suggested tools
Presentation / UI Layer	Give users and stakeholders a real interface to the agent: a query box, an evidence/path viewer, a confidence display, and a guardrail status badge (see Section 8.5).	ipywidgets (in-notebook), Streamlit, or Gradio
1 · Data & Evidence Sources	Forecast & Orders; Supplier / PO / Shipment Data; Inventory & Quality Events; Planning Notes (text)	CSV/JSON extracts, Markdown/PDF/text notes (synthetic)
2 · Ingestion & Entity Resolution	Extract entities and relationships from structured records and unstructured text; resolve aliases and duplicate mentions; assign stable business keys; MERGE/upsert into the graph.	Python, Pydantic, neo4j driver, MERGE/upsert Cypher
3 · Knowledge & Retrieval Layer	Store structured relationships in a Neo4j property graph; store embedded unstructured evidence in a vector index for semantic search; support multi-hop traversal.	Neo4j + Cypher, OpenAI/sentence-transformer embeddings, FAISS / Chroma / Neo4j vector index
4 · Agentic Reasoning Layer (OpenAI)	Classify intent, plan retrieval (graph-only, vector-only, or hybrid), call tools, and synthesize a grounded diagnosis restricted to tool-returned evidence.	OpenAI SDK, tool/function calling, optional LangChain/LangGraph
5 · Evaluation & Response Output	Score responses with an LLM-as-a-judge and task-specific metrics; track latency/cost; return confidence and evidence paths.	Custom Python metrics, RAGAS/DeepEval, OpenAI judge prompt
Cross-cutting · Governance	Protect secrets, log every tool call and evidence ID, resist prompt injection, protect privacy, and require human review for high-impact actions.	Environment-based secrets, structured logging, guardrail checks, human-in-the-loop policy

8.3 Knowledge Graph Schema

Recommended node types: Part, Product, Plant, Supplier, SupplierSite, Forecast, CustomerOrder, PurchaseOrder, Shipment, InventoryPosition, QualityEvent, LogisticsLane, SubstitutePart, ProductionPlan, CalendarPeriod

Recommended relationship types: REQUIRES_PART, BUILT_AT, SOURCED_FROM, HAS_FORECAST, HAS_ORDER, COVERED_BY_PO, SHIPPED_VIA, STOCKED_AT, BLOCKED_BY_QUALITY, MOVES_ON_LANE, HAS_SUBSTITUTE, SCHEDULED_IN, IMPACTS_PLAN, VALID_FOR_PRODUCT, OCCURS_IN_PERIOD

> **Illustrative multi-hop evidence path**
>
> Forecast / Order → Product → Required Part → Supplier / PO → Shipment → Quality Event → InventoryPosition → Plant → ProductionPlan → Substitute / Alternate Source

Unstructured evidence for vector retrieval: Supplier risk notes, planning-meeting summaries, logistics alerts, quality investigation notes, substitution approvals, procurement comments, inventory exception narratives.

8.4 Suggested Python / Open-Source Toolkit

Area	Suggested options (examples, not mandatory)
Graph database	Neo4j Python driver; Cypher; optional Neo4j native vector index
Generative AI	OpenAI Python SDK; structured outputs and tool/function calling

Area	Suggested options (examples, not mandatory)
Embeddings / retrieval	OpenAI embeddings or sentence-transformers; FAISS/Chroma or Neo4j vector index; rank_bm25 optional
Agent orchestration	Plain Python functions preferred for transparency; LangChain/LangGraph or similar open-source libraries are acceptable
Validation / schemas	Pydantic; JSON Schema; deterministic Cypher allow-lists/validators
Evaluation	Custom Python metrics; RAGAS/DeepEval or equivalent; OpenAI judge prompt with a fixed rubric
Analysis / logging	pandas; numpy; time; logging; optional matplotlib for results
UI / demo	ipywidgets (lowest effort, stays inside the notebook); Streamlit or Gradio (fast standalone app with a nicer chat UI)

8.5 Demo UI Guidance

The UI does not need to be production-grade. Given the 24-hour window, it needs to make the working system visible and credible to a stakeholder audience — pick the lightest option the team can execute well rather than the most impressive-looking one.

Option	Effort	What it gives you
ipywidgets (in-notebook)	Lowest — no new framework	A text box for the question, an output panel for the structured diagnosis, and colored status text for the guardrail badge. Everything stays inside the submitted notebook.
Streamlit (streamlit run app.py)	Low–medium — roughly 1–2 hours	A standalone page with an input box, a response panel, an evidence/path viewer, and a real colored badge component. Easy to screen-share for the stakeholder demo.
Gradio (gr.ChatInterface)	Low–medium — roughly 1–2 hours	A ready-made chat UI participants can extend with a small side panel for evidence and confidence. Good fit if the team wants a chat-first demo.

Minimum UI elements (any tool):

- **Query input** — a box where the user types or picks a natural-language question.
- **Diagnosis / answer panel** — the agent's synthesized response in plain language.
- **Evidence & path viewer** — even a simple expandable list or small table showing the evidence IDs and graph path(s) used.
- **Confidence + guardrail status badge** — a clear three-state indicator next to every response: **Grounded** (green — evidence-backed, safe to act on), **Needs Review** (amber — low confidence or partial evidence), **Blocked / Escalated** (red — guardrail triggered, human review required). This is the minimum visual pattern for showing governance state at a glance; teams may restyle the colors, but keep the three states.
- **Before/after tuning view** — even a static table comparing one metric before and after the Day-3 tuning iteration is enough to tell the evaluation story visually.

A note on the status badge

This brief uses a three-state Grounded / Needs Review / Blocked badge as the minimum way to make guardrail state visible in the UI. Teams are free to restyle it (e.g., a green/amber/red traffic light, or a two-tone pass/fail badge) as long as the three underlying states are shown.

💡 **Tip**

Pick the UI tool the team already knows best. A plain ipywidgets panel that works reliably beats a fancier Streamlit app that breaks during the live demo.

9. Specific Build Tasks

Tasks are grouped by the architecture layer they build (Section 8). Every layer must be present in the final submission.

9.1 Tasks by Architecture Layer

Layer 1–2 · Data, Ingestion & Entity Resolution

- Inventory the provided files first — list record counts, key columns, and obvious join keys before writing any ingestion code.
- Define the ontology: finalize the node and relationship types for the domain (see Section 8.3).
- Assign stable business keys / IDs to entities that can recur across files.
- Write one idempotent Cypher MERGE statement per node type before writing relationship-creation statements.
- Implement MERGE/upsert ingestion so re-running the loader does not create duplicate nodes.
- Resolve aliases and repeated text mentions before creating new graph entities.
- Add constraints and indexes for the identifiers used most often in matching and traversal.
- Preserve source_id / source_file / record_id provenance on entities or relationships where practical.
- Log a before/after node & relationship count on every ingestion run to prove duplicate-safety rather than just asserting it.

Layer 3 · Knowledge & Retrieval

- Decide a chunking strategy (by paragraph or section) for unstructured notes before embedding — don't embed whole files as one chunk.
- Chunk and embed the unstructured evidence files for the domain.
- Build and query a vector index (FAISS, Chroma, or the Neo4j native vector index).
- Store the embedding model name/version alongside each vector so retrieval stays reproducible if the model changes.
- Write 2–3 raw Cypher queries by hand first to validate the schema before asking the LLM to generate Cypher.
- Write templated Cypher — or a validated generated-Cypher approach — for the domain's common question types.
- Validate multi-hop traversal against at least three benchmark evidence paths.

> 💡 **Tip**
>
> If installing Neo4j locally is a blocker, use Neo4j Aura Free (cloud, no install) or Neo4j Desktop — both speak the same Cypher used throughout this brief.

Layer 4 · Agentic Reasoning

- Implement intent classification (diagnostic, impact-analysis, or recommendation-oriented).
- Define explicit tool/function-calling schemas for the graph-query tool and the vector-search tool, with typed arguments.
- Implement a tool router that selects graph-only, vector-only, or hybrid retrieval per question.
- Restrict evidence synthesis to what retrieval tools actually returned — no unsupported model knowledge.
- Add a fallback path: if generated Cypher fails validation, retry once with a corrected prompt or fall back to a templated query instead of erroring out.
- Cap the number of tool calls per question to avoid runaway loops.
- Return the structured JSON response contract defined in Section 9.3.

Layer 5 · Evaluation

- Build a benchmark set of at least 10–15 domain questions spanning easy/medium/hard difficulty, including one deliberately ambiguous case.
- Store benchmark questions and expected answers/evidence in a simple JSON/CSV so scoring can be re-run automatically.
- Implement LLM-as-a-judge scoring against the rubric dimensions in Section 12.4.
- Add objective/deterministic checks for at least some benchmark questions (exact entity, path, or ID matches).
- Measure and record latency and cost at each pipeline stage.
- Make — and document — at least one tuning iteration, reported as a before/after table rather than a narrative claim.

Layer 6 · Presentation / UI

- Choose one UI approach — ipywidgets, Streamlit, or Gradio — based on the team's comfort level and time remaining (see Section 8.5).
- Build a query input + response panel that calls the agent function directly; no separate backend server is needed.
- Display the evidence path(s) and source identifiers returned by the agent, not just the final text answer.
- Add a visible confidence / guardrail status badge (Grounded / Needs Review / Blocked) next to each response.
- Add a simple before/after view for at least one evaluation metric so the tuning story is visible, not just written down.
- Test the UI with one happy-path question and one ambiguous/conflicting question before the final demo.

Cross-cutting · Governance

- Keep API keys and secrets out of source files and notebooks (environment variables or a secret store).
- Log every tool call, retrieved evidence ID, and judge result for each query.

- Add guardrails against prompt injection and unsafe or out-of-policy instructions (see the full framework in Section 9.4).
- Apply the domain-specific guardrails in Section 9.5.
- Require human review before any high-impact or autonomous action.
- Write a one-page "what this agent must never do" list and test each item once before the final demo.
- Confirm no notebook, script, or log line ever prints an API key or database password.

9.2 Agent Behavior Flow

1. Validate and classify the user question — identify domain entities, time/region/product constraints, and whether the request is diagnostic, impact-analysis, or recommendation-oriented.
2. Plan retrieval — graph-only for deterministic relationship questions, vector-only for narrative evidence, or hybrid for root-cause / multi-hop questions.
3. Retrieve candidate evidence from semantic/vector search and use it to anchor or expand graph traversal.
4. Traverse Neo4j to validate relationships, connected scope, causal/temporal links, and supporting or conflicting evidence.
5. Synthesize a concise diagnosis using OpenAI, restricting claims to evidence returned by tools.
6. Return confidence/uncertainty, evidence paths, source identifiers, and human-reviewable next actions.
7. Log the query, tool calls, retrieval timing, evidence IDs, judge results, and any guardrail/escalation event.

9.3 Structured Response Contract

Every agent answer should be returned in a structured, machine-checkable format, for example:

```
{
  "diagnosis": "Evidence-grounded summary of the issue",
  "likely_causes_or_drivers": [
    {"item": "...", "confidence": 0.82, "evidence_ids": ["..."]}
  ],
  "evidence_paths": ["NodeA -> REL -> NodeB -> REL -> NodeC"],
  "affected_scope": ["products / parts / dealers / variants / plans as relevant"],
  "contradictory_or_missing_evidence": ["..."],
  "recommended_next_actions": ["human-reviewable action ..."],
  "risk_or_governance_flags": ["..."],
  "overall_confidence": 0.78
}
```

9.4 Guardrail Framework

These eight categories are the baseline every team is expected to cover, scaled to what is realistic in a 24-hour build. Section 12.1's Enterprise Governance weight is scored against this framework plus the domain-specific guardrails below.

Category	Why it matters	Minimal feasible implementation for this capstone
Input validation & prompt-injection resistance	Malicious or malformed input could hijack the agent or corrupt the graph.	A keyword/pattern denylist plus a system-prompt instruction to ignore instructions found inside retrieved data; test against 2–3 injection attempts before the demo.
Output validation / groundedness	Prevents the agent from asserting claims the evidence doesn't support.	Validate the structured JSON response against a schema (Pydantic); confirm every evidence_id referenced actually exists before returning it.
Secrets & access control	Protects credentials and any sensitive dataset fields.	Load API keys and DB credentials from environment variables/.env (never commit them); use a least-privilege Neo4j user.
Privacy / sensitive-data handling	Avoids exposing or inferring sensitive personal data.	Use only the supplied synthetic/pseudonymous dataset; block any field flagged as sensitive from being echoed verbatim.
Rate limiting & cost control	Prevents runaway OpenAI usage while testing or demoing.	Cap max tokens per call, cache repeated queries, and log token/cost per request.
Human-in-the-loop escalation	Keeps high-impact or low-confidence actions under human control.	Add a needs_human_review flag whenever overall_confidence falls below a set threshold or a safety-relevant keyword is detected.
Audit logging	Provides traceability for every decision the agent makes.	Log every query, tool call, evidence ID, and judge score with a timestamp — a flat file or local SQLite table is sufficient.
Status transparency (UI)	Lets a non-technical stakeholder see the guardrail state at a glance, without reading logs.	Show the Grounded / Needs Review / Blocked badge described in Section 8.5 next to every response in the demo UI.

> 💡 **Tip**
> OpenAI's Moderation endpoint, or a simple keyword/regex denylist, is enough for a 24-hour project — a custom-trained safety classifier is not expected.

9.5 Domain-Specific Guardrails

- The agent may recommend mitigation options but must not autonomously place orders, commit suppliers, or alter production schedules.
- Substitution recommendations must cite compatibility/approval evidence from the supplied dataset.
- The agent should separate observed facts from inferred causal links and explicitly state assumptions.
- All recommendations should include affected products/plants, evidence, constraints, and confidence.

10. Execution Plan & Team Roles

10.1 Three-Day / 24-Hour Schedule

Time	Workstream	Expected checkpoint
Day 1 — Hrs 0–1	Kickoff & data discovery	Review scenario, inspect files, define success questions and team roles.
Day 1 — Hrs 1–3	Ontology & entity strategy	Define nodes/relationships, IDs, constraints, duplicate-resolution strategy, and provenance model.
Day 1 — Hrs 3–6	Neo4j ingestion	Build the incremental loader, MERGE/upsert logic, constraints/indexes, and source linkage.
Day 1 — Hrs 6–8	Graph QA checkpoint	Run graph integrity checks and baseline Cypher questions; prove re-ingestion causes no duplicate growth.
Day 2 — Hrs 0–2	Semantic retrieval	Chunk/embed unstructured evidence; build the vector index; validate top-k retrieval.
Day 2 — Hrs 2–5	Agent workflow	Implement OpenAI-based routing/planning, the graph query tool, the vector tool, evidence synthesis, and structured outputs.
Day 2 — Hrs 5–7	Hybrid multi-hop reasoning	Answer domain benchmark questions using graph + vector evidence; add confidence and contradiction handling.
Day 2 — Hrs 7–8	Interim demo	Show one end-to-end diagnostic case and capture gaps for Day 3.
Day 3 — Hrs 0–2	Evaluation harness	Create the benchmark set, objective checks, and the LLM-as-a-judge rubric.
Day 3 — Hrs 2–4	Baseline evaluation	Measure grounding, path correctness, usefulness, failure cases, latency, and cost/usage where available.
Day 3 — Hrs 4–6	Tune & govern	Improve prompts/retrieval/query planning; add guardrails, audit logs, privacy/secrets handling, human review.
Day 3 — Hrs 6–8	Package & present	Finalize README/notebooks, results, architecture, evaluation evidence, demo story, and known limitations.

10.2 Recommended Team Split (teams of 3–5)

- **Graph/Data lead** — ontology, ingestion, deduplication, Cypher, data quality.
- **Agent/Retrieval lead** — vector search, tool routing, OpenAI integration, structured responses.
- **Evaluation/Governance lead** — benchmark set, judge, metrics, logs, guardrails, privacy/security checks.
- **Integration/Demo lead** — end-to-end notebook flow, test scenarios, documentation, final presentation.
- Teams may combine roles; every participant should understand the complete system end to end.

11. Submission Requirements

Every team submits three things. Each is graded from a different angle: the documentation shows understanding, the working project shows execution, and the stakeholder deck shows whether the team can communicate the result to a non-technical audience.

11.1 Project Documentation

A written brief (Markdown, Word, or PDF) covering:

- Problem statement and scenario recap, in the team's own words.
- Architecture diagram and a short explanation of each component choice.
- The ontology / graph schema actually used, and how it differs from the recommended schema (if at all).
- Retrieval and agent design decisions, including any fallback or tool-routing logic.
- Evaluation results — benchmark scores, judge results, and the before/after tuning comparison.
- Governance and guardrail implementation summary, mapped to the framework in Section 9.4.
- Known limitations and what would be needed for production readiness.

11.2 Working Project

- One or more .ipynb notebooks and supporting .py files that run end to end in VS Code with a Python 3.11/3.12 IPython kernel.
- A populated Neo4j graph with example Cypher queries a reviewer can run directly.
- A working demo UI (Section 8.5) a stakeholder can interact with live or via screen share — not only notebook cell output.
- Clear, reproducible setup/run instructions (README) — a reviewer should be able to start the demo without asking the team questions.

11.3 Stakeholder Presentation

A short, business-facing slide deck (8–10 slides) presenting the project to stakeholders who are not technical and were not in the room for the build. Keep it outcome-focused — what the problem was, what the agent does, and what it's worth — not a code walkthrough.

Slide	Content
1	Title & team — project name, domain, team members.
2	Business problem — the pain point, in plain business language.
3	Who it's for — the persona(s) and their day-to-day pain.
4	The solution — what the agent does, no jargon.
5	How it works — one simplified, non-technical version of the architecture.
6	Live demo — one example question and answer, shown in the UI.
7	Results — headline evaluation numbers only (judge score, latency improvement).
8	Governance & trust — how the system stays safe and auditable, in plain language.
9	Limitations & next steps — what's needed to go further.
10	Recommendation — what AGCO should do next.

> 💡 **Tip**
>
> Keep the stakeholder deck under 10 slides and leave Cypher/code screenshots out of it — save those for the project documentation.

12. Evaluation Framework & Rubric

12.1 Rubric (100%)

Category	Weight	Evidence expected
Graph Construction & Integration	30%	Entity/relationship extraction, Neo4j schema quality, incremental upserts, duplicate prevention, and reliable mapping from natural-language intent to graph retrieval.
Hybrid Retrieval & Reasoning	30%	Effective combination of vector retrieval and graph traversal; correct multi-hop reasoning; grounded evidence paths; useful diagnostic synthesis.
Evaluation & Tuning	20%	LLM-as-a-judge, task-specific metrics, benchmark questions, error analysis, latency/cost tracking, and evidence of at least one tuning iteration.
Enterprise Governance	20%	Secrets/access controls, privacy safeguards, prompt-injection resilience, auditability, source traceability, uncertainty handling, and human approval for consequential actions.

12.2 Domain-Specific Success Criteria

- Builds a connected demand-to-delivery graph with stable IDs and duplicate-safe incremental ingestion.
- Explains a shortage/disruption through a multi-hop causal path rather than isolated records.
- Combines semantic retrieval from notes with structured graph evidence — inventory, PO, supplier, and plan relationships.
- Produces ranked mitigation options with constraints and explicit evidence, and uses evaluation results to improve grounding, path accuracy, and latency.

12.3 Evaluation Guidance

Metric area	What to test
Graph/entity accuracy	Do expected entities/relationships exist? Are aliases and duplicates handled correctly?
Evidence/path correctness	Does the returned Neo4j path actually support the claimed diagnosis or driver?
Retrieval quality	Do vector results contain the relevant narrative evidence for benchmark questions?
Answer grounding	Are claims traceable to supplied evidence? Does the answer avoid unsupported model knowledge?
Diagnostic usefulness	Does the answer identify likely causes/drivers, affected scope, contradictions, and practical next actions?
Uncertainty calibration	Does the agent lower confidence or ask for review when evidence is weak or contradictory?
Governance behavior	Does it protect secrets/privacy, resist injected instructions, preserve auditability, and avoid autonomous high-impact actions?
Latency / efficiency	How long do graph, vector, and end-to-end paths take? What improved after tuning?

12.4 LLM-as-a-Judge Dimensions (score 1–5 each)

- Groundedness / factual consistency with retrieved evidence.

- Correctness of the multi-hop reasoning chain.
- Completeness of affected scope and contradictory/missing evidence.
- Usefulness and feasibility of the recommended next actions.
- Clarity of confidence, uncertainty, and governance limitations.

> 💡 **Tip**
>
> Ask the LLM-as-a-judge to return a JSON object matching a fixed schema (score per dimension plus a short justification) so scoring can be parsed automatically instead of read by hand.

> **Important**
>
> A high judge score does not replace deterministic checks. At least some benchmark questions should validate exact entities, relationships, evidence IDs, or expected path elements programmatically.

13. Dataset Pack Blueprint

The final capstone dataset is generated separately by the program team. Files should load locally from the VS Code/Jupyter environment and ingest into Neo4j without depending on external enterprise systems.

DOWNLOAD DATASET FOLDER FROM HERE

13.1 Recommended Files

Recommended file / folder	Purpose
products_bom.csv	Products and their required parts/BOM relationships.
parts.csv	Part master and compatibility metadata.
suppliers.csv	Supplier/source entities and attributes.
supplier_capacity.csv	Synthetic capacity/commitment data by period.
demand_forecast.csv	Forecast demand by product/period/region.
customer_orders.csv	Synthetic actual/committed demand signals.
purchase_orders.csv	Open/closed PO evidence and dates.
shipments.csv	Shipment status, ETA, quantity, logistics links.
inventory_positions.csv	On-hand, reserved, safety stock, plant/location balances.
quality_events.csv	Supplier/part quality holds and disposition status.
plants_and_production_plans.csv	Plant/product/build schedule relationships.
substitution_rules.csv	Approved substitute/compatibility constraints.
logistics_lanes.csv	Synthetic lane/lead-time/risk metadata.
unstructured_supply_notes/ (Markdown/PDF/text)	Narrative supplier/planning/logistics evidence.

Dataset design principle

Include enough deliberate ambiguity, duplicate mentions, conflicting signals, and cross-file relationships to force entity resolution, hybrid retrieval, and multi-hop reasoning. The dataset should not be solvable with a single lookup table.


Demand-to-Delivery Diagnostic Agent — Data Flow at a Glance
All 13 datasets + the unstructured notes folder, staged from raw file to structured diagnosis — and what happens when new data lands.
STAGE 0 Data Sources — every file below is raw input; nothing answers a question alone
REFERENCE & MASTER DATA
parts.csv 15.1k rows
products_bom.csv 21.6k rows
suppliers.csv 15.2k rows
substitution_rules.csv 16.0k rows
DEMAND SIDE
demand_forecast.csv 21.6k rows
customer_orders.csv 30.3k rows
SUPPLY & LOGISTICS EXECUTION
supplier_capacity.csv 28.0k rows
purchase_orders.csv 27.0k rows
shipments.csv 21.1k rows
logistics_lanes.csv 15.2k rows
QUALITY, PLANTS & PRODUCTION
quality_events.csv 16.0k rows
inventory_positions.csv 24.0k rows
plants_and_production_plans.csv 18.0k rows
UNSTRUCTURED EVIDENCE
unstructured_supply_notes/ 59 .md files
supplier risk notes · quality investigations · planning summaries · substitution approvals · procurement comments · inventory exceptions · logistics alerts
STAGE 1 Ingestion & Entity Resolution — every file passes through the same pipeline before touching the graph
Structured CSVs
Parse rows, assign stable business keys (part_id , plant_id , po_id ...)
Resolve aliases/duplicates before creating nodes (e.g. -ALT part rows, flagged duplicate suppliers)
MERGE/upsert into Neo4j — idempotent, so re-running a load never inflates counts
Unstructured Notes (.md)
Chunk each note; embed each chunk (model/version recorded)
Keep related_*_id front-matter as anchors back to graph node IDs
STAGE 2 Knowledge & Retrieval Layer — where datasets actually combine, inside Neo4j
parts.csv + products_bom.csv
(:Product)-[:REQUIRES_PART]->(:Part)
what a product needs, and how much
parts/suppliers + purchase_orders.csv
(:Supplier)-[:SOURCED_FROM]->(:Part)-[:COVERED_BY_PO]->(:PO)
who supplies it, is an order open/late
purchase_orders + shipments + logistics_lanes
(:PO)-[:SHIPPED_VIA]->(:Shipment)-[:MOVES_ON_LANE]->(:Lane)
is it moving, is the lane high-risk
quality_events + parts/suppliers
(:Part)-[:BLOCKED_BY_QUALITY]->(:QualityEvent)
is any supply on hold
inventory_positions + production_plans
(:Part)-[:STOCKED_AT]->(:Plant)-[:SCHEDULED_IN]->(:Plan)
enough on hand for the next build
substitution_rules.csv
(:Part)-[:HAS_SUBSTITUTE]->(:Part)
what's approved instead, and its limits
demand_forecast + customer_orders
(:Product)-[:HAS_FORECAST]->(:HAS_ORDER]->(...)
did real demand back the forecast
unstructured_supply_notes/
vector index, anchored via related_*_id
narrative "why", not in any CSV
STAGE 3 Agentic Reasoning Layer — decides which Stage 2 patterns and/or notes a question actually needs
INTENT CLASSIFY diagnostic / impact / recommendation
TOOL ROUTER graph-only / vector-only / hybrid
EVIDENCE SYNTHESIZER restricted to tool-returned facts
RESPONSE BUILDER fills the structured contract
OUTCOME Structured Diagnosis — never a bare text answer
diagnosis • likely_causes_or_drivers[confidence, evidence_ids] • evidence_paths • affected_scope • contradictory_or_missing_evidence • recommended_next_actions • risk_or_governance_flags • overall_confidence
Golden thread in this dataset:
SC-417 at Plant P2 — a late PO, an unrelated quality hold, and a forecast uplift combine into one shortage, with a limited-compatibility substitute and a cross-plant transfer as partial mitigations.
When new data arrives later
No second system. New rows/notes re-enter at Stage 1, resolve against existing entities, and MERGE into the same Neo4j graph + same vector index as additional nodes, relationships, or vectors — never a separate store.
Any of: parts.csv · products_bom.csv · suppliers.csv · supplier_capacity.csv · demand_forecast.csv · customer_orders.csv · purchase_orders.csv · shipments.csv · inventory_positions.csv · quality_events.csv · plants_and_production_plans.csv · substitution_rules.csv · logistics_lanes.csv · unstructured_supply_notes/*.md

13.2 Example Diagnostic / Benchmark Questions

1. Why is Part SC-417 projected to create a shortage at Plant P2 in the next planning window?
2. Which demand, supplier, shipment, inventory, and quality events contribute to the risk, and in what order?
3. Which products or build plans are exposed if the shortage is not mitigated?
4. Are there approved substitutes, alternate suppliers, or inventory transfers that can reduce the risk, and what constraints apply?
5. Which mitigation option gives the best balance of coverage, lead time, and operational risk based on the supplied evidence?

14. Final Demo Storyboard & Completion Checklist

14.1 Demo Storyboard

1. Show the domain problem and one benchmark question.
2. Run or explain incremental ingestion; prove duplicate-safe graph behavior.
3. Visualize or query the relevant Neo4j neighborhood/path.
4. Ask the agent the benchmark question and show which retrieval tools it uses.
5. Show the final structured diagnosis with evidence paths, source IDs, confidence, and recommended next actions.
6. Show one ambiguous/conflicting case where the agent correctly expresses uncertainty or requests human review.
7. Present evaluation scores before/after tuning and one latency/quality trade-off.
8. Close with governance controls, known limitations, and what would be needed for productionization.

14.2 Optional Stretch Goals

- Add scenario comparison for forecast uplift/downside and resulting shortage exposure.
- Create a graph-based supplier concentration or single-point-of-failure analysis.
- Add temporal path reasoning to explain how risk changed across planning periods.
- Build a human-approved action-plan export for planners.

14.3 Completion Checklist

☐ Neo4j schema, constraints, and duplicate-safe ingestion are working.

☐ Vector index is built from unstructured evidence and retrieval is validated.

☐ OpenAI agent can call graph and vector tools and produce structured answers.

☐ At least one multi-hop diagnostic path is demonstrated end to end.

☐ Benchmark/evaluation suite and LLM-as-a-judge are reproducible.

☐ Latency and at least one tuning iteration are documented.

☐ Secrets, audit logs, privacy, prompt-injection handling, uncertainty, and human review are addressed.

☐ README, architecture, notebooks, and final demo are complete.